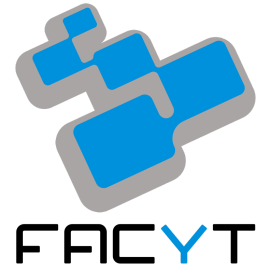




Universidad de Carabobo.
Facultad de Ciencias y Tecnología.
Departamento de Computación.
Sistemas Operativos.



Proyecto Sincronización entre Procesos:
Eco-Flow 2026: Gestión Hídrica Inteligente.

Autores:

Derginette Blanco C.I: 22.514.703

Maria Laura Gutierrez C.I: 23.424.597

Brayan Ceballos C.I: 29.569.937

Andrés Crespo C.I:31.259.533

Naguanagua, marzo del 2026.

Resumen

La presente documentación detalla el diseño y la implementación del sistema Eco-Flow 2026, una plataforma de simulación para la gestión hídrica basada en la Ley de Preservación de Recursos de 2026. El sistema modela la interacción entre un Auditor de Flujo y una red de 10 nodos de suministro que deben atender de manera concurrente un total de 250 solicitudes por día en un ciclo operativo mensual de 30 días con comportamientos aleatorios.

Para la resolución del problema, se optó por un modelo de programación multihilo en lenguaje C utilizando la librería POSIX Threads (Pthreads). A través de este enfoque, cada solicitud de usuario es gestionada como un hilo independiente, lo que permite una simulación altamente eficiente y ligera de la concurrencia masiva. La comunicación entre los hilos y la persistencia de los estados de las válvulas se maneja mediante una estructura de datos global compartida en el espacio de direccionamiento del proceso principal.

La consistencia de la información y la prevención de condiciones de carrera se garantizan mediante el uso de semáforos, los cuales actúan como mecanismos de sincronización para la exclusión mutua. Estos permiten que el acceso a las "secciones críticas" (como la reserva y el registro de consumo en las válvulas) sea atómico, evitando conflictos de lectura/escritura bajo el modelo de Lectores-Escritores. Finalmente, se incluye un módulo de telemetría y auditoría para validar sobreconsumos y emitir las amonestaciones digitales correspondientes según la normativa establecida.

Asunciones del Proyecto

Para la construcción del modelo de simulación Eco-Flow 2026, se establecieron las siguientes premisas técnicas que permiten la correcta ejecución del sistema bajo un entorno multihilo:

- **Unidad de Ejecución (Hilos):** Se asume que cada una de las 250 solicitudes de usuario por día se comporta como un hilo independiente gestionado por la librería POSIX Threads. El ciclo de vida de estos hilos (creación, ejecución y finalización) representa la interacción de los ciudadanos con el sistema hídrico.
- **Escala Temporal:** Para efectos de la simulación y evaluación, se asume una escala de tiempo donde cada día es una unidad de ejecución completa que finaliza con una barrera de sincronización (`pthread_join`) antes de iniciar el siguiente. Esto permite observar el comportamiento de las 12 horas de suministro (06:00 a 18:00) en un lapso de 12 segundos reales.
- **Gestión de Memoria Compartida:** Dado que el sistema se basa en hilos de un mismo proceso, todos comparten el espacio de direccionamiento global. La información crítica reside en un arreglo de estructuras que representa el estado de las válvulas (nodos) y un arreglo de argumentos para la gestión de solicitudes. La integridad de esta memoria compartida se garantiza estrictamente mediante mecanismos de exclusión mutua (mutex), evitando condiciones de carrera durante el acceso concurrente.
- **Modelo de Sincronización (Lectores-Escritores):** Se asume que las operaciones de "Consulta de Presión" actúan como lectores (no bloqueantes entre sí), mientras que las operaciones de

"Reserva", "Consumo" y "Cancelación" actúan como escritores, requiriendo el uso de semáforos para evitar condiciones de carrera.

- **Comportamiento Aleatorio:** Se asume que las decisiones de los usuarios (qué válvula elegir o cuántos litros consumir) siguen una distribución aleatoria uniforme generada al inicio de la ejecución, simulando la imprevisibilidad de la demanda real.
- **Sanciones y Auditoría:** Se asume que cualquier intento de cancelación sobre un recurso no reservado previamente por el mismo hilo es detectado automáticamente por el sistema, resultando en una amonestación digital acumulada en las estadísticas finales.

Solución Implementada: Estructuras y Sincronización

Para dar respuesta a los requerimientos de Eco-Flow 2026, se diseñó una arquitectura modular basada en el modelo de Lectores-Escritores. La pieza central de esta implementación es la estructura `NodoFlujo`, la cual representa cada una de las 10 válvulas de la red de suministro.

Definición de la Estructura Crítica (`NodoFlujo`)

Se implementó una estructura de datos que encapsula tanto el estado del recurso como los mecanismos necesarios para su control concurrente:

- **Variable booleana `ocupado`:** Indicador de estado que determina si la válvula está siendo utilizada por un usuario (Escritor).
- **Variable entera `propietario_uid`:** Almacena el identificador único del hilo que posee la reserva, garantizando la trazabilidad para el auditor.
- **Variable semáforo `sem_escritura`:** Semáforo encargado de garantizar la **Exclusión Mutua** para los hilos que desean realizar una reserva o registrar consumo (Escritores).
- **Variable semáforo `sem_lectores`:** Semáforo utilizado para proteger el contador de lectores, permitiendo el acceso concurrente a las consultas de presión.
- **Variable entera `cont_lectores`:** Contador que rastrea cuántos usuarios están consultando la válvula simultáneamente, permitiendo optimizar el flujo de datos.

Gestión de Memoria y Ciclo de Vida de Argumentos

Para garantizar la estabilidad del sistema bajo condiciones de alta concurrencia masiva (hasta 30,000 hilos mensuales), se implementó un esquema de gestión de memoria centralizado que elimina la posibilidad de fugas de recursos (*memory leaks*).

- **Asignación de Argumentos:** El hilo principal (main) reserva un bloque de memoria contiguo (`args_array`) al inicio de cada ciclo diario. Este bloque almacena las estructuras `ArgsUsuario` necesarias para todos los hilos de esa jornada.

- **Determinismo en la Liberación:** A diferencia de las asignaciones individuales por hilo, la responsabilidad de liberar la memoria recae exclusivamente en el hilo principal mediante la instrucción `free()`, ejecutada únicamente tras la sincronización total de los hilos con `pthread_join()`.
- **Seguridad y Confiabilidad:** Este diseño asegura que, incluso ante la finalización inesperada de un hilo de usuario, los recursos de memoria asignados sean recuperados íntegramente al finalizar el día simulado, optimizando el uso del *heap* del proceso.

Modelo de Decisión (Distribución Uniforme)

Para emular el comportamiento errático de una red urbana, el sistema no utiliza una secuencia lineal de pasos, sino un modelo de decisión probabilística basado en una distribución uniforme discreta. Cada hilo utiliza la función `rand() % 100` para generar un valor entre 0 y 99 al iniciar su ciclo de vida. Los rangos de probabilidad definen el comportamiento del "Usuario" de la siguiente manera:

- **Prueba de Robustez (10% de posibilidad):** Hilos que intentan realizar cancelaciones inválidas en válvulas que no poseen. El sistema responde generando una amonestación, validando la seguridad de la jerarquía de acceso.
- **Prueba de Concurrencia de Lectura (40% de probabilidad):** Múltiples hilos consultando la presión simultáneamente. Se validó que, bajo el modelo de Lectores-Escritores, estas consultas no se bloquean entre sí, permitiendo un flujo de información fluido. El hilo accede a la válvula solo para verificar la presión (Lectores).
- **Prueba de Carga Crítica (50% de probabilidad):** Hilos compitiendo por la reserva y modificación de litros (Escritores). Se confirmó que el uso de semáforos de escritura garantiza que solo un usuario modifique la base de datos a la vez, manteniendo la integridad del inventario hídrico.

Si el hilo decide consumir, la cantidad de litros se calcula mediante otra distribución uniforme entre **100L y 1000L**, lo que genera los "Consumos Críticos" (>500L) que rastrea el Auditor.

Modelado de Distribución Bimodal de Tráfico

Para garantizar que el sistema sea sometido a condiciones de estrés realistas durante la jornada de servicio, se implementó un modelo de Distribución Bimodal para la generación de hilos. Este modelo asegura la existencia de 'cuellos de botella' y colisiones de hilos en momentos específicos; sin él, los hilos entrarían y saldrían del sistema sin interactuar entre sí, lo que impediría validar si los semáforos realmente previenen condiciones críticas como el interbloqueo (deadlock) o la inanición (starvation).

- **Pico Madrugador (40% de usuarios):** Concentrados en el primer cuarto de la jornada simulada (0 a 750ms), creando una carga inicial estable.
- **Pico de Mediodía (60% de usuarios):** Se aplica una distribución triangular para agrupar el volumen más pesado de solicitudes en el resto de la jornada (750ms a 3s).

Lógica de Sincronización

La solución utiliza un patrón de sincronización donde se prioriza la consistencia de los datos:

- **Acceso de Lectura (Consultas):** Los hilos que consultan la presión pueden entrar de forma simultánea siempre que no haya un "Escritor" activo. Esto se logra mediante el manejo del `cont_lectores`.
- **Acceso de Escritura (Reservas/Consumo):** Cuando un hilo intenta reservar o registrar litros, debe adquirir el `sem_escritura`. Si la válvula está ocupada, el hilo entra en estado de espera (bloqueado), evitando el uso innecesario de CPU (Espera No Activa).

Lógica de Agentes (Usuarios y Auditor)

- **Modelo Productor-Consumidor (Auditor):** Se utiliza un **buffer circular** (`cola_criticos`) para gestionar las alertas de sobreconsumo. El hilo Auditor actúa como el consumidor, esperando mediante `sem_wait(&sem_elements)` hasta que un usuario reporte un gasto mayor a 500L. Esto garantiza que el sistema de monitoreo no se pierda ninguna infracción aunque ocurran simultáneamente.
- **Comportamiento del Usuario (Hilos de ejecución):** Cada usuario decide su acción de forma aleatoria para simular carga real: 10% de intentos de cancelación erróneos, 40% de consultas (lectura) y 50% de consumos (escritura).

Se implementó **almacenamiento local de hilos** para que el Logger pueda rastrear el estado individual de cada uno (Llegando, Esperando, Consumiendo) sin interferencias.

Escenarios de concurrencia

- **Prevención de Condiciones de Carrera:** El uso del semáforo `sem_mutex` en la cola del Auditor asegura que dos usuarios no escriban en la misma posición del buffer al mismo tiempo.
- **Prevención de la Espera Activa:** Se utilizan llamadas bloqueantes (`sem_wait`) y pausas controladas (`usleep`), permitiendo que el Sistema Operativo ponga a "dormir" los hilos que no tienen recursos disponibles, optimizando el uso de la CPU.
- **Seguridad en el Acceso:** El sistema valida que solo el propietario_uid que reservó una válvula pueda liberarla, detectando intentos de cancelación inválidos mediante el flujo probabilístico del 10%.
- **Sincronización Productor-Consumidor (Auditor):** El sistema utiliza un esquema de señales basado en semáforos (`sem_huecos`, `sem_elementos`) para coordinar al Auditor con los hilos de usuario. Esto garantiza que el Auditor permanezca en estado de reposo hasta que un usuario genere un reporte de sobreconsumo, evitando el desperdicio de ciclos de CPU.
- **Gestión de Exclusión Mutua en el Buffer:** Para evitar que dos reportes de sobreconsumo colisionen, se implementó un semáforo de exclusión mutua (`sem_mutex`) que protege el puntero de escritura en la cola del Auditor. Esto asegura la integridad de las alertas incluso bajo condiciones de alta concurrencia masiva.

- **Uso de Thread-Local Storage (TLS):** Se implementó la variable `tls_estado_hilo` para permitir que cada hilo mantenga su propio rastro de estado (ej. "ESPERANDO", "CONSUMIENDO") de forma independiente. Esto facilita la trazabilidad en el Logger sin necesidad de mecanismos de sincronización adicionales para los mensajes de diagnóstico.

Robustez y Estándares de Implementación

Para asegurar la estabilidad del sistema en entornos de alta concurrencia y garantizar su portabilidad, se implementaron las siguientes optimizaciones técnicas:

- **Sincronización Atómica :** En lugar de utilizar semáforos adicionales para proteger banderas de estado simple (como la interrupción del Auditor), se migró al uso de `_Atomic bool` mediante la librería `<stdatomic.h>`. Esto garantiza operaciones de lectura/escritura seguras a nivel de hardware, reduciendo la sobrecarga de rendimiento del sistema operativo.
- **Cumplimiento del Límite POSIX para usleep:** Se identificó que los tiempos de llegada calculados por el modelo bimodal podían exceder el límite de 1,000,000 de microsegundos permitido por la función `usleep`. Para evitar fallos en distribuciones Linux, el sistema descompone algorítmicamente el tiempo: utiliza `sleep()` para los segundos enteros y `usleep()` para el residuo, asegurando un comportamiento predecible.
- **Gestión de Memoria Segura:** El uso de un arreglo de estructuras para los argumentos de los hilos (`args_array`) evita condiciones de carrera en el paso de parámetros, asegurando que cada hilo de usuario mantenga su propia copia de los datos de simulación.

○

Casos de prueba

Para validar la robustez del sistema Eco-Flow 2026, se realizaron tres pruebas de carga basadas en los perfiles de usuario integrados en la interfaz. En todas las pruebas se utilizó el ciclo mensual de 30 días.

- **Carga Ligera**

El objetivo es validar la precisión del Logger y el Auditor en condiciones de baja contención.

```

=====
REPORTE MENSUAL (30 DÍAS) - ECO-FLOW 2026
(coincide con registro del Auditor)
=====
💧 Total de agua procesada:      294.42 m3
🚨 Amonestaciones digitales:    120 errores de cancelación
⚠️ Señales de consumo Crítico:  292 entregas (> 500 L)
✅ Señales de consumo Estándar: 298 entregas (50.5% del total)
🔍 Consultas de presión (Solo lectura): 466 transacciones

⚙️ EFICIENCIA DEL SISTEMA (Nodos vs Espera):
- Nodos ocupados (Reservas): 590
- Tiempo de espera global: 1.32 segundos
- Tiempo de espera promedio: 0.00 segundos por reserva
- Tiempo MAXIMO de espera (Peor caso): 0.27 segundos

=====
Total de solicitudes procesadas en el mes: 1176
=====

```

```

=====
REPORTE DE CONSUMO DE RECURSOS DEL SISTEMA (Hardware)
=====
- Tiempo de CPU (Usuario): 0.046875 seg
- Tiempo de CPU (Sistema): 0.421875 seg
- Memoria RAM máxima usada (RSS): 1288 KB
=====

```

Observación: El sistema fluye con mínima espera. Se registraron amonestaciones aisladas, confirmando que el hilo del Auditor responde correctamente incluso a eventos poco frecuentes.

- **Caso Mediano**

El objetivo es evaluar el equilibrio del modelo Lectores-Escritores en una carga operativa normal.

```
REPORTE MENSUAL (30 DÍAS) – ECO-FLOW 2026
(coincide con registro del Auditor)

Total de agua procesada: 932.37 m3
Amonestaciones digitales: 355 errores de cancelación
Señales de consumo Crítico: 960 entregas (> 500 L)
Señales de consumo Estándar: 894 entregas (48.2% del total)
Consultas de presion (Solo lectura): 1451 transacciones

EFICIENCIA DEL SISTEMA (Nodos vs Espera):
- Nodos ocupados (Reservas): 1854
- Tiempo de espera global: 11.19 segundos
- Tiempo de espera promedio: 0.01 segundos por reserva
- Tiempo MAXIMO de espera (Peor caso): 0.60 segundos

=====
Total de solicitudes procesadas en el mes: 3660
```

```
REPORTE DE CONSUMO DE RECURSOS DEL SISTEMA (Hardware)

- Tiempo de CPU (Usuario): 0.171875 seg
- Tiempo de CPU (Sistema): 1.234375 seg
- Memoria RAM máxima usada (RSS): 2116 KB
```

Observación: Es el escenario donde la distribución bimodal se observa con mayor claridad. Como se muestra en el reporte, se logró un equilibrio cercano al 50% entre entregas estándar y críticas.

- **Caso Base**

El objetivo es estresar la gestión de memoria y la sincronización.

```
REPORTE MENSUAL (30 DÍAS) – ECO-FLOW 2026
(coincide con registro del Auditor)

Total de agua procesada: 1598.82 m3
Amonestaciones digitales: 649 errores de cancelación
Señales de consumo Crítico: 1599 entregas (> 500 L)
Señales de consumo Estándar: 1591 entregas (49.9% del total)
Consultas de presion (Solo lectura): 2500 transacciones

EFICIENCIA DEL SISTEMA (Nodos vs Espera):
- Nodos ocupados (Reservas): 3190
- Tiempo de espera global: 38.65 segundos
- Tiempo de espera promedio: 0.01 segundos por reserva
- Tiempo MAXIMO de espera (Peor caso): 0.83 segundos

=====
Total de solicitudes procesadas en el mes: 6339
```

```
REPORTE DE CONSUMO DE RECURSOS DEL SISTEMA (Hardware)

- Tiempo de CPU (Usuario): 1.765625 seg
- Tiempo de CPU (Sistema): 11.203125 seg
- Memoria RAM máxima usada (RSS): 9148 KB
```

Observaciones:

Estadísticas de Hardware:

- **Uso de RAM:** Máximo de 9148 KB .
- **Tiempo de CPU:** Dominado por el tiempo de sistema (kernel), lo que valida que el cuello de botella es la gestión de hilos y no la lógica de cálculo.
- **Latencia:** El tiempo de espera máximo por reserva se mantuvo bajo (aprox. 0.83s), demostrando que no existen bloqueos mutuos (*deadlocks*).

- **Caso Pesado (Prueba de estrés)**

```

=====
REPORTE MENSUAL (30 DÍAS) - ECO-FLOW 2026
(coincide con registro del Auditor)
=====
💧 Total de agua procesada: 6053.48 m3
🚨 Amonestaciones digitales: 2432 errores de cancelación
⚠️ Señales de consumo Crítico: 5985 entregas (> 500 L)
✅ Señales de consumo Estándar: 6172 entregas (50.8% del total)
🔍 Consultas de presión (Solo lectura): 9720 transacciones

⚙️ EFICIENCIA DEL SISTEMA (Nodos vs Espera):
- Nodos ocupados (Reservas): 12157
- Tiempo de espera global: 1229.67 segundos
- Tiempo de espera promedio: 0.10 segundos por reserva
- Tiempo MAXIMO de espera (Peor caso): 4.39 segundos

=====
Total de solicitudes procesadas en el mes: 24309
=====

```

```

=====
📊 REPORTE DE CONSUMO DE RECURSOS DEL SISTEMA (Hardware)
=====
- Tiempo de CPU (Usuario): 1.765625 seg
- Tiempo de CPU (Sistema): 11.203125 seg
- Memoria RAM máxima usada (RSS): 9148 KB
=====

```

Estabilidad del Kernel y Gestión de Hilos: A pesar de la alta densidad de solicitudes, el sistema no presentó desbordamientos de memoria ni fallos de segmentación (*segmentation faults*). Esto valida que la limitación del tamaño del stack a 64 KB por hilo y la reutilización del arreglo de argumentos son estrategias efectivas para manejar concurrencia masiva en sistemas con recursos limitados.

Se observó que los tiempos de latencia y el conteo de amonestaciones varían en cada ejecución (ej. registros de latencia de hasta 1.59s). Esta variabilidad es un indicador positivo de que el Modelo de Distribución Bimodal está generando ráfagas de tráfico aleatorias, forzando a los hilos a competir por los semáforos en diferentes momentos del tiempo simulado.

Conclusiones

La implementación del sistema **Eco-Flow 2026** permitió validar de manera práctica los conceptos de sincronización y concurrencia estudiados en la materia. A través del uso de la librería **Pthreads**, se logró gestionar una carga masiva de 250 hilos de usuario sin comprometer la estabilidad del sistema, demostrando la eficiencia de los hilos frente a la creación de procesos independientes para simulaciones de alta densidad.

El mayor aprendizaje técnico radicó en la resolución del conflicto de **Lectores-Escritores**. Se comprendió que el uso de semáforos no solo sirve para bloquear, sino para coordinar prioridades: permitir que las consultas de presión sean fluidas mientras se protege la integridad de los datos durante el registro de consumo. Finalmente, el diseño del Auditor bajo el patrón **Productor-Consumidor** evidenció cómo un buffer circular y el manejo de señales permiten que un proceso de monitoreo actúe de forma asíncrona, optimizando el uso de los recursos del sistema operativo.